9

ПРОЕКТИРОВАНИЕ ПОИСКОВОГО РОБОТА

В этой главе мы сосредоточимся на интересной классической задаче, которую можно встретить на интервью по проектированию ИТ-систем, — создании поискового робота.

Поисковый робот еще называют веб-пауком или веб-краулером. Он широко используется в поисковых системах для обнаружения нового или обновленного контента в Сети. Это могут быть веб-страницы, изображения, видео, PDF-файлы и т. д. Сначала поисковый робот собирает несколько веб-страниц, а затем проходит по всем ссылкам, которые они содержат, чтобы собрать новый контент. Пример этого процесса показан на рис. 9.1.

Поисковый робот применяется для множества разных задач.

- Индексация в поисковой системе. Это самый распространенный сценарий использования. Робот собирает веб-страницы, чтобы создать локальный индекс для поисковой системы. Например, в поисковой системе Google эту роль играет Googlebot.
- Веб-архивация. Это процесс сбора информации с веб-страниц для дальнейшего хранения и использования. Например, архивацией веб-сайтов занимаются многие национальные библиотеки. В качестве известных примеров можно привести Библиотеку Конгресса США [1] и Веб-архив ЕС [2].
- Извлечение веб-данных. Сверхбыстрое развитие интернета создает беспрецедентную возможность для сбора информации. Извлечение веб-данных помогает собирать в Сети ценные сведения. Например, ведущие финансовые фирмы используют поисковые роботы для загрузки стенограмм собраний акционеров и ежегодных отчетов для анализа ключевых инициатив компании.

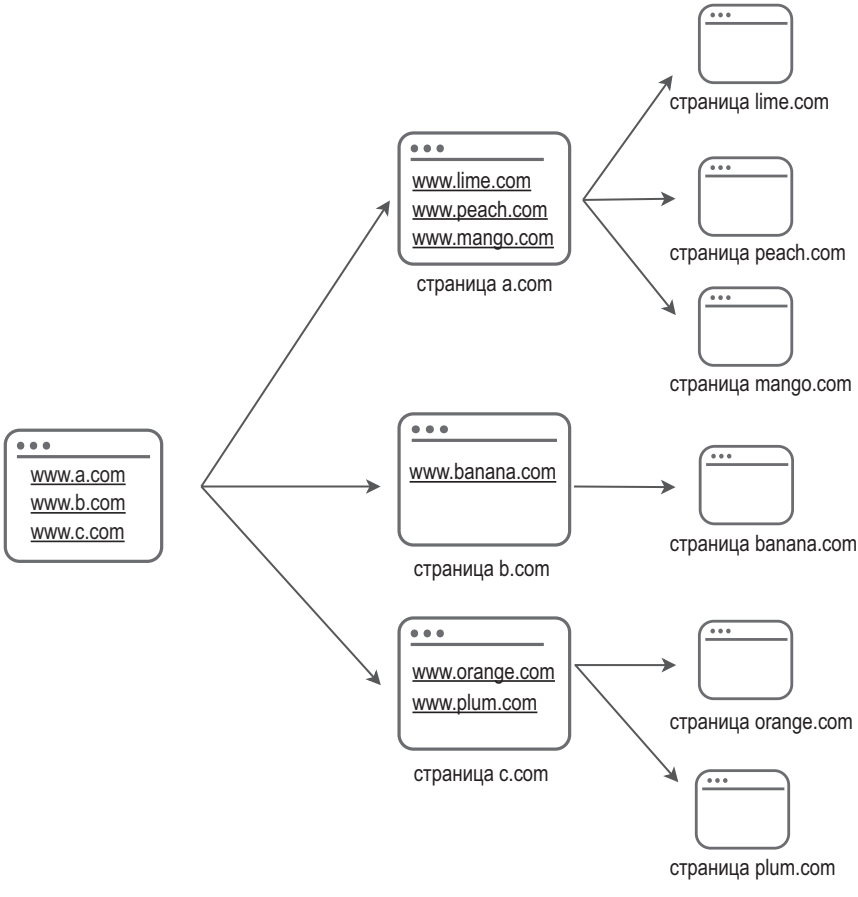


Рис. 9.1

- Веб-мониторинг. Поисковые роботы помогают отслеживать нарушения авторских прав и незаконное использование торговых марок в интернете. Например, Digimarc [3] таким образом ищет пиратские копии и отчеты.

Сложность разработки поискового робота зависит от того, какой масштаб он должен поддерживать. Это может быть как скромный школьный проект, который можно закончить за пару часов, так и гигантская система, требующая постоянного внимания со стороны целой команды инженеров. Поэтому сначала нужно определиться с масштабом и функциями, которые должны поддерживаться.

ШАГ 1: ПОНЯТЬ ЗАДАЧУ И ОПРЕДЕЛИТЬ МАСШТАБ РЕШЕНИЯ

Поисковый робот работает по простому принципу:

1. На вход подается список URL-адресов, после чего загружаются соответствующие веб-страницы.
2. Из веб-страниц извлекаются URL-адреса.
3. Новые URL-адреса добавляются в список для дальнейшей загрузки. Эти три шага повторяются заново.

Ограничивается ли работа поискового робота этим простым алгоритмом? Не совсем. Спроектировать широкомасштабный поисковый робот очень сложно. Вряд ли кому-то удастся это сделать во время интервью. Прежде чем переходить к архитектуре, необходимо задать вопросы о требованиях и о масштабе системы.

Кандидат: «Каково основное назначение поискового робота? Он используется для индексации в поисковой системе, сбора данных или для чего-то другого?»

Интервьюер: «Для индексации в поисковой системе».

Кандидат: «Сколько веб-страниц поисковый робот собирает в месяц?»

Интервьюер: «1 миллиард страниц».

Кандидат: «Какого рода контент мы ищем? Только HTML или другие ресурсы вроде PDF-файлов и изображений?»

Интервьюер: «Только HTML».

Кандидат: «Следует ли учитывать недавно добавленные или отредактированные веб-страницы?»

Интервьюер: «Да».

Кандидат: «Нужно ли хранить собранные HTML-страницы?»

Интервьюер: «Да. Срок хранения — 5 лет».

Кандидат: «Что делать с веб-страницами, содержимое которых дублируется?»

Интервьюер: «Их следует игнорировать».

Это лишь некоторые из тех вопросов, которые можно задать интервьюеру. Необходимо разобраться в требованиях и прояснить непонятные моменты. Даже если вас попросят спроектировать такой простой продукт, как поисковый робот, у вас с интервьюером может быть разное понимание того, что он должен собой представлять.

Помимо функциональности, которую нужно согласовать с интервьюером, необходимо учитывать следующие характеристики поискового робота.

- Масштабируемость. Интернет огромен. В нем миллиарды веб-страниц. Поисковый робот должен быть чрезвычайно эффективным и использовать параллельные вычисления.
- Устойчивость. Интернет полон ловушек. Вам постоянно будет встречаться некорректный HTML-код, неотзывчивые серверы, сбои, вредоносные ссылки и т. д. Поисковый робот должен справляться со всеми этими пограничными случаями.
- Вежливость. Поисковый робот не должен отправлять веб-сайту слишком много запросов за короткий промежуток времени.
- Расширяемость. Система должна быть гибкой, чтобы для поддержки новых видов контента не приходилось вносить масштабные изменения. Например, если в будущем нам понадобится собирать графические файлы, это не должно привести к переработке всей системы.

Приблизительные оценки

Следующие оценки основаны на множестве предположений, поэтому вы должны убедиться в том, что вы с интервьюером правильно поняли друг друга.

- Предположим, что каждый месяц загружается 1 миллиард веб-страниц.
- QPS: 1 000 000 000 / 30 дней / 24 часа / 3600 секунд = ~400 страниц в секунду.
- Пиковый показатель QPS = 2 * QPS = 800.
- Предположим, что средний размер веб-страницы составляет 500 Кб.

- 1 миллиард страниц * 500 Кб = 500 Тб в месяц для хранения. Если вы плохо ориентируетесь в единицах измерения данных, перечитайте раздел «Степень двойки» в главе 2.
- Если данные хранятся на протяжении пяти лет, 500 Тб * 12 месяцев * 5 лет = 30 Пб. Для контента, собранного за пять лет, нужно хранилище размером 30 Пб.

ШАГ 2: ПРЕДЛОЖИТЬ ОБЩЕЕ РЕШЕНИЕ И ПОЛУЧИТЬ СОГЛАСИЕ

Определившись с требованиями, мы переходим к общим архитектурным вопросам. На рис. 9.2 изображена предлагаемая архитектура, вдохновленная исследованиями [4] [5].

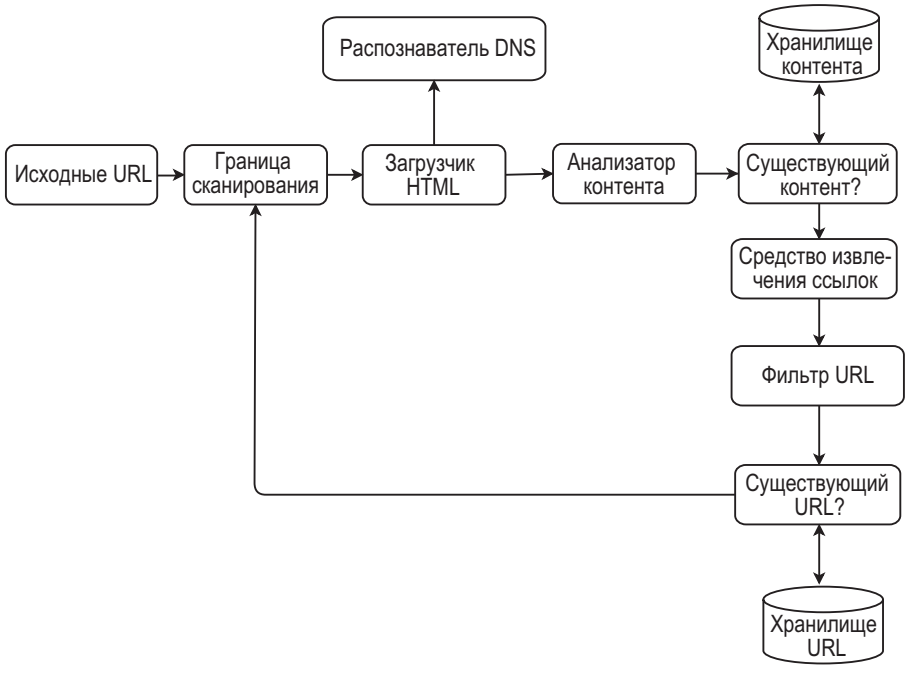


Рис. 9.2

Сначала мы исследуем каждый компонент архитектуры, чтобы понять его функции, а затем шаг за шагом рассмотрим принцип работы поискового робота.

Исходные URL-адреса

В процессе поиска в качестве отправной точки используются исходные URL-адреса. Например, чтобы перебрать все веб-страницы на университетском веб-сайте, нам, очевидно, следует начать с доменного имени университета.

Для обхода всего интернета нам нужно творчески подойти к выбору исходных URL-адресов. Хороший исходный URL-адрес позволит перебрать как можно большее количество ссылок. Общая стратегия — разделить пространство адресов на отдельные части. Первый предложенный подход основан на местоположении, так как популярность тех или иных веб-сайтов может зависеть от страны. Исходные URL-адреса также можно выбирать по тематике. Например, адресное пространство можно разделить на покупки, спорт, медицину и т. д. Выбор исходных URL-адресов зависит от разных факторов. От вас не ожидают идеального решения. Просто размышляйте вслух.

Граница сканирования

Большинство современных поисковых роботов делят контент на уже загруженный и ожидающий загрузки. Компонент, хранящий URL-адреса, предназначенные для загрузки, называется границей сканирования. Его можно считать очередью вида «первым пришел, первым ушел». Углубленную информацию об этом компоненте можно найти в разделе, посвященном углубленному проектированию.

Загрузчик HTML

Этот компонент загружает веб-страницы из интернета. URL-адреса этих веб-страниц предоставляются границей сканирования.

Распознаватель DNS

Чтобы загрузить веб-страницу, URL сначала нужно перевести в IP-адрес. Для этого загрузчик HTML может обратиться к распознавателю DNS. Например, по состоянию на 5.3.2019 URL www.wikipedia.org преобразуется в IP-адрес 198.35.26.96.

Анализатор контента

После загрузки веб-страницы ее нужно проанализировать: вдруг у нее некорректный HTML-код, который вызовет проблемы и только займет место в хранилище? Если разместить анализатор контента на одном сервере с поисковым роботом, это замедлит процесс сбора данных. В связи с этим он имеет вид отдельного компонента.

Существующий контент?

Как показывает онлайн-исследование [6], 29 % от всех веб-страниц являются дубликатами, что может привести к повторному сохранению одного и того же контента. Мы используем структуру данных «Существующий контент?», чтобы избежать дублирования информации и сократить время обработки. Она помогает обнаруживать новый контент, который система уже сохраняла. Но это медленный подход, занимающий много времени, особенно если речь идет о миллиардах веб-страниц. Для эффективного выполнения этой задачи следует сравнивать не сами веб-страницы, а их хеши [7].

Хранилище контента

Это система хранения HTML. Ее выбор зависит от таких факторов, как тип и размер данных, частота доступа, время жизни и т. д. Используется как диск, так и память.

- Большая часть контента хранится на диске, так как набор данных слишком большой и не умещается в памяти.
- Востребованный контент хранится в памяти для снижения латентности.

Средство извлечения ссылок

Этот компонент анализирует HTML-страницы и извлекает из них ссылки. Пример этого процесса показан на рис. 9.3. Относительные пути преобразуются в абсолютные URL-адреса путем добавления префикса https://en.wikipedia.org.

```
<html class="client-nojs" lang="en" dir="ltr">
   <head>
      <meta charset="UTF-8"/>
      <title>Wikipedia, the free encyclopedia</title>
   </head>
   <body>
      <li><a href="/wiki/Cong_Weixi" title="Cong Weixi">Cong Weixi</a></li>
      <li><a href="/wiki/Kay_Hagan" title="Kay Hagan">Kay Hagan</a></li>
      <li><a href="/wiki/Vladimir_Bukovsky" title="Vladimir Bukovsky">Vladimir Bukovsky</a></li>
      <li><a href="/wiki/John_Conyers" title="John Conyers">John Conyers</a></li>
   </body>
</html>
```

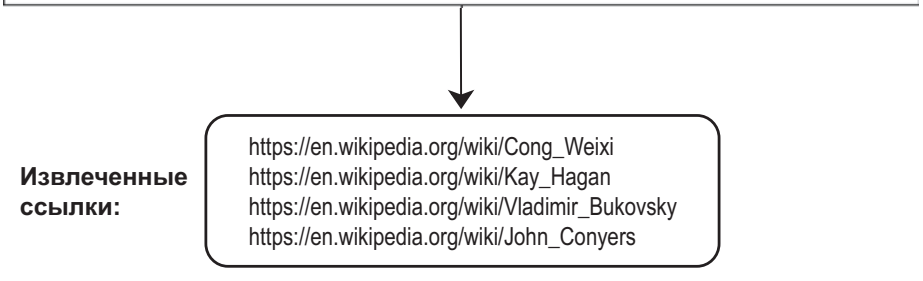


Рис. 9.3

Фильтр URL-адресов

Фильтр URL-адресов отклоняет определенные типы контента, расширения файлов, ссылки на страницы с ошибками и URL-адреса, занесенные в черный список.

Существующий URL-адрес?

«Существующий URL-адрес?» — это структура данных, которая отслеживает URL-адреса, которые уже посещались или находятся в границе сканирования. Это помогает избежать повторного открытия одного и того же URL-адреса, которое чревато повышенной нагрузкой на сервер и потенциальными бесконечными циклами.

Для реализации этого компонента обычно применяют такие методики, как фильтр Блума и хеш-таблица. Мы не станем в них углубляться. Дополнительную информацию можно найти в справочных материалах [4] [8].

Хранилище URL-адресов

Это хранилище уже посещенных URL-адресов.

Итак, мы рассмотрели каждый отдельный компонент. Теперь мы соберем их воедино и обсудим принцип работы системы в целом.

Принцип работы поискового робота

Чтобы как следует объяснить каждый этап рабочего процесса, мы добавили на диаграмму архитектуры порядковые номера (рис. 9.4).

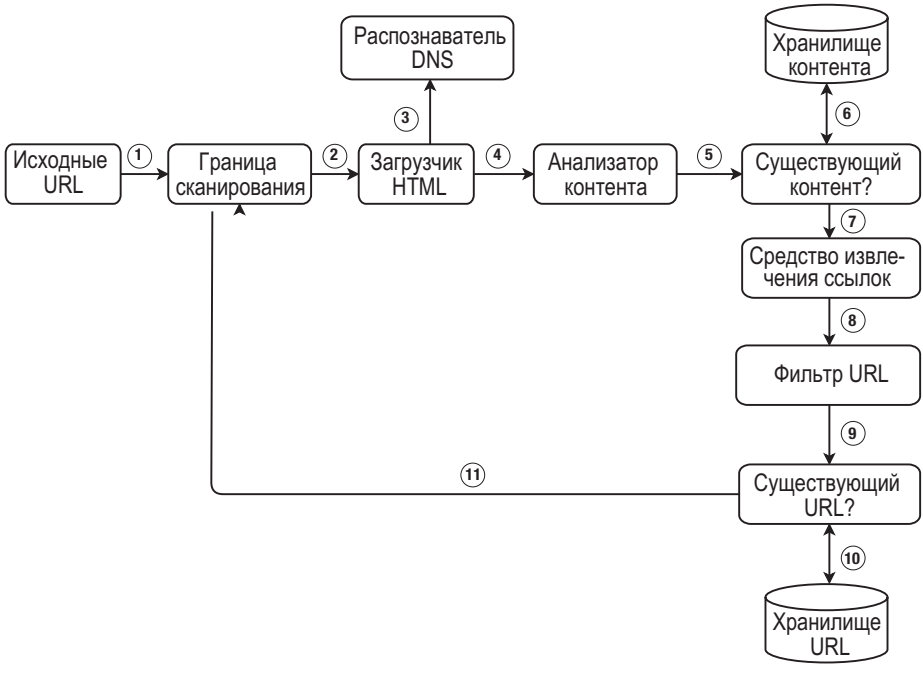


Рис. 9.4

- Шаг 1. Добавляем исходные URL-адреса в границу сканирования.
- Шаг 2. Загрузчик HTML берет список URL-адресов из границы сканирования.

- Шаг 3. Загрузчик HTML получает соответствующие IP-адреса от распознавателя DNS и начинает загрузку.
- Шаг 4. Анализатор контента разбирает HTML-страницы и проверяет их корректность.
- Шаг 5. После разбора и проверки контент передается компоненту «Существующий контент?».
- Шаг 6. Компонент «Существующий контент?» проверяет, есть ли данная HTML-страница в хранилище:
  - если есть, это означает, что этот контент находится по другому URL-адресу и мы его уже обработали. В этом случае HTML-страница отклоняется;
  - если нет, система еще не обрабатывала этот контент, поэтому он передается средству извлечения ссылок.
- Шаг 7. Из HTML-страниц извлекаются ссылки.
- Шаг 8. Извлеченные ссылки передаются фильтру URL-адресов.
- Шаг 9. После фильтрации ссылки передаются компоненту «Существующий URL-адрес?».
- Шаг 10. Компонент «Существующий URL-адрес?» проверяет, находится ли URL в хранилище. Если да, то он уже обрабатывался и больше ничего делать не нужно.
- Шаг 11. Если URL-адрес еще не обрабатывался, он добавляется в границу сканирования.

ШАГ 3: ПОДРОБНОЕ ПРОЕКТИРОВАНИЕ

До сих пор мы обсуждали общие аспекты архитектуры. Теперь рассмотрим подробно самые важные составные компоненты и характеристики:

- границу сканирования;
- загрузчик HTML;
- устойчивость;
- расширяемость;
- обнаружение и отклонение проблемного контента.

DFS и BFS

Интернет можно считать направленным графом, вершинами которого являются веб-страницы, а ребрами — гиперссылки (URL-адреса). Процесс поиска можно рассматривать как обход направленного графа, начиная с корневой вершины. Для этого существует два распространенных алгоритма: поиск в глубину (deep-first search, DFS) и поиск в ширину (breadth-first search, BFS). При этом DFS обычно является не самым удачным вариантом, так как граф может быть очень глубоким.

Поисковые роботы, как правило, используют алгоритм BFS, который реализуется с помощью очереди FIFO. URL-адреса достаются из очереди в том порядке, в котором они добавлялись. Но у этой реализации есть две проблемы.

- Большинство ссылок, размещенных на одной веб-странице, указывают на один и тот же сетевой узел. На рис. 9.5 все ссылки на странице внутренние, из-за чего поисковый робот вынужден

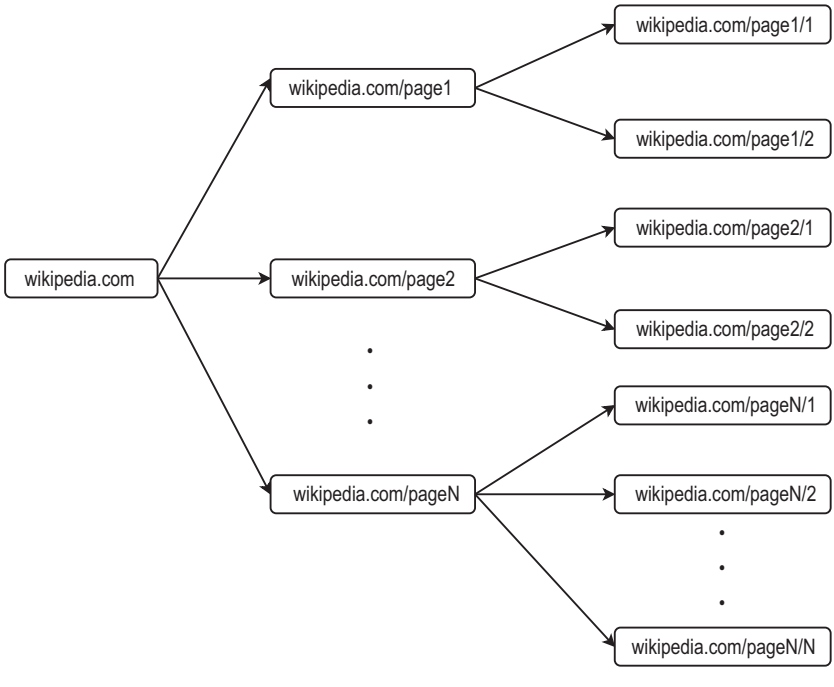


Рис. 9.5

обрабатывать URL-адреса в одном домене (wikipedia.com). Если он попытается загрузить веб-страницы параллельно, серверы Википедии будут завалены запросами. Это «невежливо».

- Стандартная версия BFS не учитывает приоритет URL-адреса. В интернете миллиарды веб-страниц, и не все они имеют одинаковый уровень качества и значимости. Поэтому приоритет обработки URL-адресов может зависеть от их рейтинга, популярности, частоты обновления и т. д.

Граница сканирования

Эти проблемы помогает решить граница сканирования. Это структура данных, хранящая URL-адреса, которые нужно загрузить. Она играет важную роль в соблюдении вежливости, расстановке приоритетов и обеспечении актуальности страниц. Несколько исследований, посвященных границе сканирования и заслуживающих вашего внимания, упоминается в справочных материалах [5][9]. Их результаты приводятся ниже.

Вежливость

В целом поисковый робот не должен отправлять серверу слишком много запросов за короткий промежуток времени. Такое поведение считается невежливым и даже может быть воспринято как DoS-атака. Например, если не предусмотреть никаких ограничений, поисковый робот может отправлять одному и тому же веб-сайту тысячи запросов в секунду и тем самым перегрузит серверы.

Основная идея соблюдения вежливости заключается в последовательной загрузке страниц с одного сервера. Между загрузками можно сделать задержку. Чтобы это реализовать, доменные имена веб-сайтов нужно привязывать к потокам (рабочим узлам) загрузки. У каждого потока-загрузчика есть отдельная очередь FIFO, и все URL-адреса он достает из нее. На рис. 9.6 показан механизм, который делает наш поисковый робот вежливым.

- Маршрутизатор очереди. Следит за тем, чтобы каждая очередь (b1, b2, ... bn) содержала URL-адреса только с одним доменным именем.
- Таблица связывания. Привязывает каждое доменное имя к очереди.

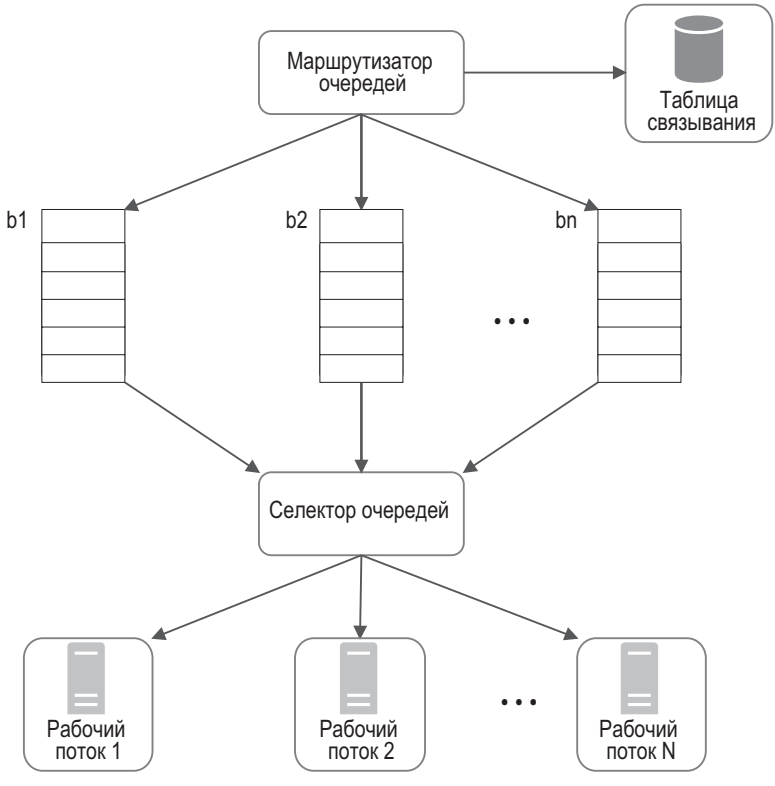


Рис. 9.6

Таблица 9.1

Доменное имя	Очередь
wikipedia.com	b1
apple.com	b2
…	…
nike.com	bn

- Очереди FIFO b1, b2, ..., bn. Каждая очередь содержит URL-адреса только с одним доменным именем.
- Селектор очередей. Каждый рабочий поток привязан к очереди FIFO, и все URL-адреса загружает только из нее. За логику выбора подходящей очереди отвечает селектор очередей.

- Рабочие потоки от 1 до N. Рабочий поток последовательно загружает веб-страницы из одного и того же сервера. Между загрузками можно предусмотреть задержку.

Приоритет

Случайное сообщение на форуме, посвященном продукции Apple, по своей значимости отличается от сообщений, опубликованных на официальном сайте этой компании. Несмотря на общее для них ключевое слово Apple, поисковому роботу лучше начинать с последнего.

Мы назначаем URL-адресам приоритеты в зависимости от их ценности. При этом можно учитывать PageRank [10], популярность веб-сайта, частоту обновления и т. д. За это отвечает средство расстановки приоритетов. Более детальную информацию на эту тему можно найти в справочных материалах [5] [10].

Механизм, назначающий приоритеты URL-адресам, показан на рис. 9.7.

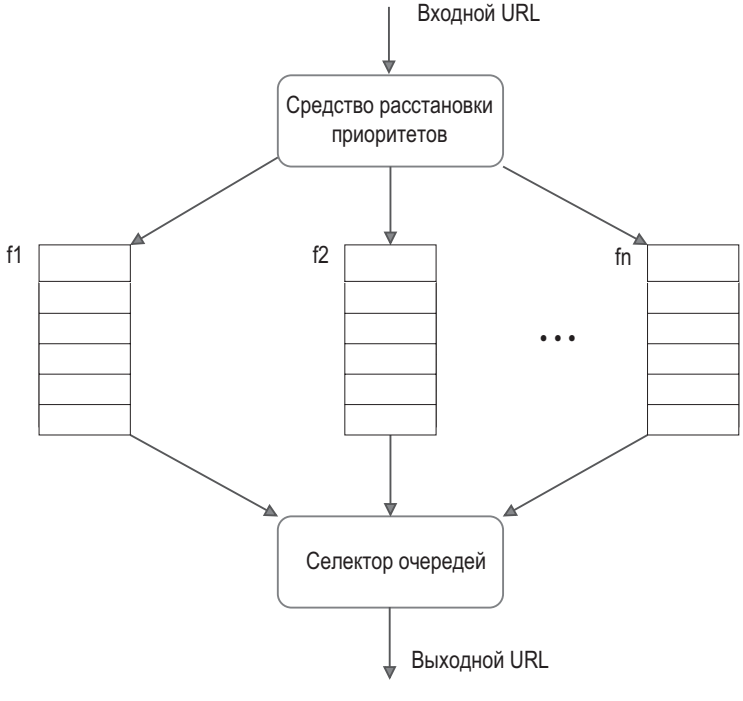


Рис. 9.7

- Средство расстановки приоритетов. Принимает на вход URL-адреса и вычисляет их приоритеты.
- Очереди с f1 по fn. Каждой очереди назначается приоритет. Чем выше приоритет, тем больше вероятность того, что очередь будет выбрана.
- Селектор очередей. Выбирает произвольную очередь, отдавая предпочтение очередям с повышенным приоритетом.

На рис. 9.8 показано, как устроена граница сканирования. Она состоит из двух модулей:

- лицевые очереди: отвечают за расстановку приоритетов;
- тыльные очереди: отвечают за соблюдение вежливости.

Актуальность

Веб-страницы постоянно добавляются, удаляются и редактируются. Поисковый робот должен периодически проходиться по уже загруженным веб-страницам, чтобы поддерживать набор данных в актуальном состоянии. Для повторного перебора всех URL-адресов нужно много времени и ресурсов. Существует две стратегии оптимизации этого процесса:

- учитывать историю обновлений веб-страниц;
- назначать URL-адресам приоритеты, чтобы в первую очередь (и чаще) загружать важные страницы.

Хранилище для границы сканирования

В настоящих поисковых системах количество URL-адресов на границе сканирования может достигать сотен миллионов [4]. Хранение всего этого в памяти плохо сказывается как на надежности, так и на масштабируемости. На диске тоже лучше не хранить все подряд, так как он медленный и легко может стать узким местом поискового робота.

Мы выбрали гибридный поход. Большая часть URL-адресов находится на диске, поэтому размер хранилища не составляет проблемы. Чтобы избежать накладных расходов, связанных с чтением и записью на диск, мы храним в памяти буфер для добавления/удаления данных из очереди. Содержимое буфера периодически сбрасывается на диск.

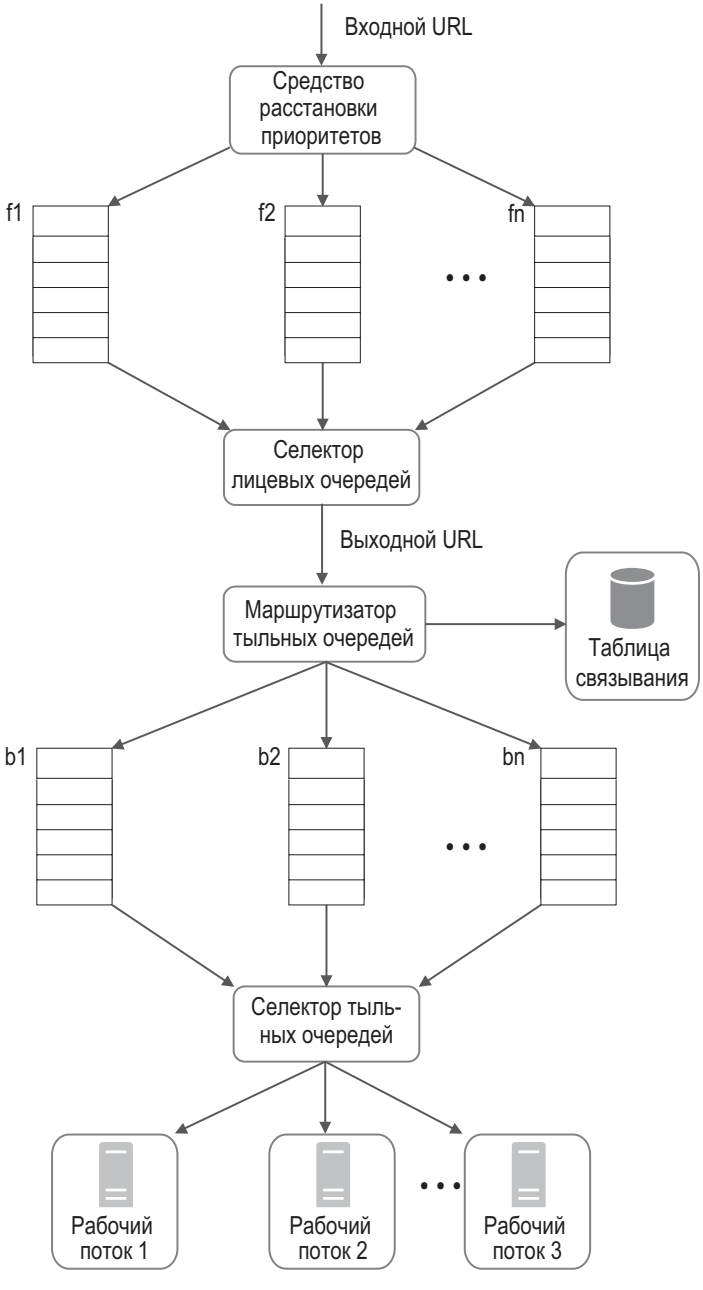

Входной URL
Средство расстановки приоритетов
f1
f2
fn
Селектор лицевых очередей
Выходной URL
Маршрутизатор тыльных очередей
Таблица связывания
b1
b2
bn
Селектор тыль-ных очередей
Рабочий поток 1
Рабочий поток 2
Рабочий поток 3

Рис. 9.8

Загрузчик HTML

Этот компонент загружает веб-страницы из интернета по протоколу HTTP. Прежде чем его обсуждать, рассмотрим стандарт исключений для роботов.

Robots.txt

Файл robots.txt (так называемый стандарт исключений для роботов) позволяет веб-сайтам взаимодействовать с поисковыми системами. В нем можно указать страницы, которые позволено загружать поисковым роботам. Прежде чем начинать обход веб-сайта, поисковый робот должен сначала проверить соответствующий файл robots.txt и следовать его правилам.

Чтобы не загружать robots.txt повторно, мы кэшируем его содержимое. Файл периодически загружается и сохраняется в кэш. Вот фрагмент, взятый из https://www.amazon.com/robots.txt. Некоторые директории, такие как creatorhub, закрыты для робота Google.

```
User-agent: Googlebot
Disallow: /creatorhub/*
Disallow: /rss/people/*/reviews
Disallow: /gp/pdp/rss/*/reviews
Disallow: /gp/cdp/member-reviews/
Disallow: /gp/aw/c r/
```

Еще одним важным аспектом загрузчика HTML является оптимизация производительности, речь о которой пойдет дальше.

Оптимизация производительности

Ниже перечислены оптимизации загрузчика HTML.

1. **Распределенный поиск**. Для обеспечения высокой производительности процесс поиска должен быть распределен по разным серверам, каждый из которых работает в несколько потоков. Адресное пространство делится на части, чтобы каждый загрузчик отвечал за отдельное подмножество URL-адресов. Пример распределенного поиска показан на рис. 9.9.
2. **Кэширующий распознаватель DNS**. Распознаватель DNS является узким местом поискового робота, так как многие интерфейсы DNS синхронны по своей природе, и DNS-запросы могут занимать какое-то время. Время DNS-ответа варьируется от 10 мс до 200 мс. Когда поток поискового робота обращается к DNS-серверу, другие



Рис. 9.9

потоки блокируются, пока не завершится первый запрос. В качестве эффективного метода повышения скорости, который позволит избежать частого обращения к DNS, можно использовать кэш DNS. Наш кэш хранит соответствия между доменными именами и IP-адресами и периодически обновляется с помощью заданий cron.

3. **Локальность.** Распределите серверы поискового робота географически. Чем ближе они к серверам веб-сайта, тем короче время загрузки. Локальность относится к большинству компонентов системы: серверам поискового робота, кэшу, очереди, хранилищу и т. д.
4. **Короткое время ожидания.** Некоторые серверы отвечают медленно или вовсе не отвечают. Чтобы не ждать их слишком долго, указывается максимальное время ожидания. Если сервер не ответит в отведенный промежуток времени, поисковый робот остановит задание и попробует загрузить какие-то другие страницы.

Устойчивость

Наряду с оптимизацией производительности важную роль играет устойчивость. Вот несколько подходов к улучшению устойчивости системы.

- Согласованное хеширование. Оно помогает распределить нагрузку между загрузчиками и позволяет добавлять и удалять серверы загрузки. Подробнее об этом — в главе 5 «Согласованное хеширование».

- Сохранение состояния и данных поискового робота. Чтобы уберечься от сбоев, состояния и данные поискового робота записываются в систему хранения. В случае нарушения работы поисковый робот можно легко перезапустить, загрузив сохраненные состояния и данные.
- Обработка исключений. В крупномасштабных системах ошибки являются неизбежным и распространенным явлением. Поисковый робот должен как следует справляться с исключениями, не нарушая работу системы.
- Проверка корректности данных. Это важная мера для предотвращения системных ошибок.

Расширяемость

Почти всегда по мере развития системы разработчики стремятся сделать ее гибкой и обеспечить поддержку новых типов контента. Поисковый робот можно расширить за счет подключения новых модулей. На рис. 9.10 показано, как это делается.

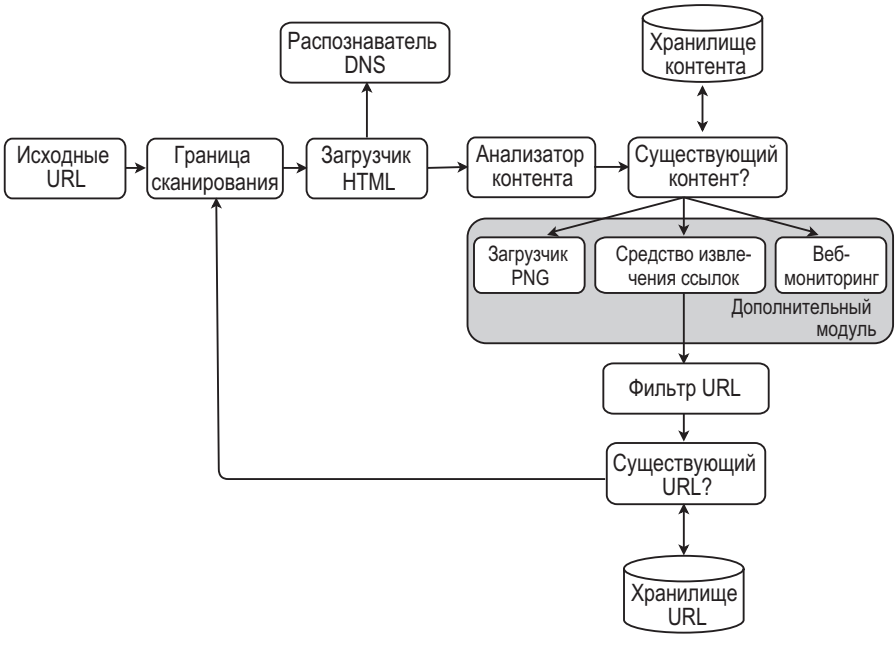


Рис. 9.10

- Загрузчик PNG — это подключаемый модуль для загрузки PNG-файлов.
- Модуль веб-мониторинга подключается для отслеживания и предотвращения нарушений авторских прав и незаконного использования товарных знаков в интернете.

Обнаружение и отклонение проблемного контента

В этом разделе обсуждается обнаружение и предотвращение загрузки лишнего, бессмысленного или вредоносного контента.

1. **Лишний контент.** Как говорилось выше, около 30 % всех страниц являются дубликатами. Для их обнаружения можно использовать хеши или контрольные суммы [11].
2. **Ловушки.** Речь идет о веб-страницах, которые заставляют поискового робота входить в бесконечный цикл. Например, вот ссылка на бесконечно глубокую структуру директорий: www.spidertrapexample.com/foo/bar/foo/bar/foo/bar/....

   Таких ловушек можно избежать, если установить максимальную длину URL-адресов. Но вообще, для их обнаружения не существует какого-то универсального метода. Веб-сайты с ловушками можно легко определить по необычно большому количеству страниц, которые они содержат. Сложно написать алгоритм, который будет автоматически избегать таких ловушек; это можно сделать вручную, а затем либо занести соответствующие веб-сайты в черный список, либо применить собственные фильтры для URL-адресов.
3. **Бессмысленные данные.** Некоторый контент несет в себе мало смысла, а порой его нет вообще. Это относится к рекламе, листингам кода, спаму и т. д. Такой контент бесполезен для поисковых роботов, и его по возможности следует отклонять.

ШАГ 4: ПОДВЕДЕНИЕ ИТОГОВ

Мы начали эту главу с обсуждения характеристик хорошего поискового робота — масштабируемости, вежливости, расширяемости и устойчивости. Затем мы предложили вариант архитектуры и рассмотрели ее

ключевые компоненты. Создание масштабируемого поискового робота — нетривиальная задача, ведь интернет огромен и полон ловушек. И хотя нам удалось охватить множество аспектов, без внимания осталось еще много важного.

- Генерация страниц на стороне сервера. Многие веб-сайты генерируют ссылки на лету, используя JavaScript, AJAX и т. д. Если мы будем загружать и разбирать веб-страницы напрямую, нам не удастся извлечь динамически сгенерированные ссылки. Чтобы решить эту проблему, перед разбором веб-страницы мы генерируем ее на сервере [12].
- Фильтрация нежелательных страниц. Поскольку емкость хранилища и ресурсы поискового робота ограничены, будет полезно использовать компонент для борьбы со спамом, который будет фильтровать низкокачественные и рекламные страницы [13] [14].
- Репликация и сегментирование базы данных. Такие методики, как репликация и сегментирование, позволяют улучшить доступность, масштабируемость и надежность уровня данных.
- Горизонтальное масштабирование. Для крупномасштабного обхода загрузкой должны заниматься сотни или даже тысячи серверов. Главное, чтобы они не хранили свое состояние.
- Доступность, согласованность и надежность. Это ключевые характеристики успеха любой крупной системы. Мы подробно обсудили их в главе 1.
- Аналитика. Сбор и анализ данных — важная часть системы и ключевой элемент ее оптимизации.

Поздравляем, вы проделали длинный путь и можете гордиться собой. Отличная работа!

СПРАВОЧНЫЕ МАТЕРИАЛЫ

[1] Библиотека Конгресса США: https://www.loc.gov/websites/

[2] Веб-архив ЕС: http://data.europa.eu/webarchive

[3] Digimarc: https://www.digimarc.com/products/digimarc-services/piracy-intelligence

[4] Heydon A., Najork M. Mercator: A scalable, extensible web crawler World Wide Web, 2 (4) (1999), pp. 219-229

[5] By Christopher Olston, Marc Najork: Web Crawling. http://infolab.stanford.edu/~olston/publications/crawling_survey.pdf

[6] 29% Of Sites Face Duplicate Content Issues: https://tinyurl.com/y6tmh55y

[7] Rabin M.O., et al. Fingerprinting by random polynomials Center for Research in Computing Techn., Aiken Computation Laboratory, Univ. (1981)

[8] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," Communications of the ACM, vol. 13, no. 7, pp. 422–426, 1970.

[9] Donald J. Patterson, Web Crawling: https://www.ics.uci.edu/~lopes/teaching/cs221W12/slides/Lecture05.pdf

[10] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Technical Report, Stanford University, 1998.

[11] Burton Bloom. Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 13(7), pages 422–426, July 1970.

[12] Google Dynamic Rendering: https://developers.google.com/search/docs/guides/dynamic-rendering

[13] T. Urvoy, T. Lavergne, and P. Filoche, "Tracking web spam with hidden style similarity," in Proceedings of the 2nd International Workshop on Adversarial Information Retrieval on the Web, 2006.

[14] H.-T. Lee, D. Leonard, X. Wang, and D. Loguinov, "IRLbot: Scaling to 6 billion pages and beyond," in Proceedings of the 17th International World Wide Web Conference, 2008.